

Understanding Log Files and Debugging Errors in Yosys

INTRODUCTION:

In Synthesis Yosys convert high-level RTL (Verilog) code into a gate-level netlist. During this process, any errors in the source code and commands are detected and reported in the form of log messages. Understanding these logs is essential for debugging and ensuring a clean synthesis flow. This experiment demonstrates how to intentionally introduce errors in Verilog code, observe Yosys log outputs, identify the source of errors, and correct the RTL to achieve a successful synthesis result.

OBJECTIVE:

After completing this lab, you will be able to:

- Understand how Yosys generates log messages during synthesis.
- Learn to locate syntax and semantic errors in the log output.
- Practice debugging common RTL coding mistakes (e.g., missing semicolons, commas, brackets).
- Gain confidence in using Yosys logs as a primary tool for error detection and correction.
- Develop systematic debugging skills by iterating between faulty RTL, log analysis, and corrected RTL.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir practice`
- 1.4: Open project directory: `cd practice`

Step 2: Create Design file

- 2.1: Open gvim editor for design with command: `gvim design.v`

RTL Design Code:

```
module counter_4bit_up (  
    input wire clk,  
    input wire reset  
    output reg [3:0] count    // ERROR 1: Missing comma  
)  
  
always @(posedge clk or posedge reset) begin
```

```
if (reset)

    count <= 4'b0000;

else

    count <= count + 1    // ERROR 2: Missing semicolon

end

endmodule
```

```
module counter_4bit_up (
    input wire clk,
    input wire reset
    output reg [3:0] count    // ERROR 1: Missing comma
)
always @(posedge clk or posedge reset) begin
    if (reset)
        count <= 4'b0000;
    else
        count <= count + 1    // ERROR 2: Missing semicolon
end
endmodule
```

2.1 : Save the file using command- :wq!

Step 3: Create a Script For Synthesis

3.1 vim script.js

3.2 write the script

3.2 save this file using wq! & list using - ls

```
coe-12@ip-10-100-19-29:~/Desktop/Lab/lab/Understanding_log_file$ ls
design.v  osu018_stdcells.lib  script.js
coe-12@ip-10-100-19-29:~/Desktop/Lab/lab/Understanding_log_file$
```

// the script we created is not an executable file so we have to convert it into an executable file

3.3 Convert This File Into Executable File

Using - chmod +777 script.js

```
coe-12@ip-10-100-19-29:~/Desktop/Lab/lab/Understanding_log_file$ chmod +777 script.js
coe-12@ip-10-100-19-29:~/Desktop/Lab/lab/Understanding_log_file$ ls
design.v  osu018_stdcells.lib  script.js
coe-12@ip-10-100-19-29:~/Desktop/Lab/lab/Understanding_log_file$
```

Step 4: Synthesize the Design with Yosys

Run the script

yosys -s script.js | tee synth.log // the tee command will save a log file of the yosys session

```
coe-12@ip-10-100-19-29:~/Desktop/Lab/lab/Understanding_log_file$ yosys -s script.js | tee synth.log
design.v:4: ERROR: syntax error, unexpected TOK_OUTPUT, expecting ',', '=' or ')'
```

```
/-----\
| yosys -- Yosys Open SYnthesis Suite
|
| Copyright (C) 2012 - 2019 Clifford Wolf <clifford@clifford.at>
|
| Permission to use, copy, modify, and/or distribute this software for any
| purpose with or without fee is hereby granted, provided that the above
| copyright notice and this permission notice appear in all copies.
|
| THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES
| WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF
| MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR
| ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
| WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN
| ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF
| OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.
|-----\
```

Yosys 0.9 (git sha1 1979e0b)

-- Executing script file `script.js' --

1. Executing Verilog-2005 frontend: design.v

```
coe-12@ip-10-100-19-29:~/Desktop/Lab/lab/Understanding_log_file$
```

4.1: Analyze & Fix The Errors

○ Error 1 :

- In this case the error is in the rtl because the tool is stopped just after reading the Verilog
- **Yosys produced the following error message in the log:**

```
design.v:4: ERROR: syntax error, unexpected TOK_OUTPUT, expecting ',',
or '=' or ')'
```

- Use `cat synth.log` to open the log file analyze the error

• Analyze The Error ;

- File: design.v
- Line: 4
- Problem: Yosys encountered `output` in a place where it expected **a comma, =, or)**.
- Meaning: There is likely a **missing comma or semicolon** in the port declaration list.

- **Fix the Error ;**

- Open the Verilog(rtl) with gvim command
- Check and fix the errors – fix semicolon and comma in line 4 and 11
- Save the rtl with wq!

```
module counter_4bit_up (  
    input wire clk,  
    input wire reset,  
    output reg [3:0] count  
)  
  
always @(posedge clk or posedge reset) begin  
    if (reset)  
        count <= 4'b0000;  
    else  
        count <= count + 1;  
end  
  
endmodule  
  
i
```

- Re-Run The Script

```
6.7. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module \counter_4bit_up..
Removed 2 unused cells and 140 unused wires.
<suppressed ~3 debug messages>

6.8. Executing OPT_EXPR pass (perform const folding).
Optimizing module counter_4bit_up.

6.9. Rerunning OPT passes. (Maybe there is more to do..)

6.10. Executing OPT_MUXTREE pass (detect dead branches in mux trees).
Running muxtree optimizer on module \counter_4bit_up..
Creating internal representation of mux trees.
No muxes found in this module.
Removed 0 multiplexer ports.

6.11. Executing OPT_REDUCE pass (consolidate $*mux and $reduce_* inputs).
Optimizing cells in module \counter_4bit_up.
Performed a total of 0 changes.

6.12. Executing OPT_MERGE pass (detect identical cells).
Finding identical cells in module \counter_4bit_up'.
Removed a total of 0 cells.

6.13. Executing OPT_RMDFF pass (remove dff with constant values).

6.14. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module \counter_4bit_up..

6.15. Executing OPT_EXPR pass (perform const folding).
ERROR: Can't open liberty file `Osu018_stdcells.lib': No such file or directory
Optimizing module counter_4bit_up.

6.16. Finished OPT passes. (There is nothing left to do.)

7. Executing DFFLIBMAP pass (mapping DFF cells to sequential cells from liberty file).
coe-12@ip-10-100-19-29:~/Desktop/Lab/Lab/Understanding_log_file$
```

○ Error 2 :

```
6.15. Executing OPT_EXPR pass (perform const folding).
ERROR: Can't open liberty file `Osu018_stdcells.lib': No such file or directory
Optimizing module counter_4bit_up.
```

- In this case the error is in the library path because the tool stopped executing while reading library
- Yosys produced the following error message in the log:


```
11. Executing DFFLIBMAP pass (mapping DFF cells to sequential cells from
liberty file).
ERROR: Can't open liberty file `Osu018_stdcells.lib': No such file or
directory
```
- Use `cat synth.log` to open the log file analyze the error

- **Analyze The Error ;**

- **Check if the file exists**

- Make sure `osu018_stdcells.lib` is actually present.
- If not, download or copy it from the OSU 180nm PDK (or whichever library you're using).
- Correct the path in the script

- **Fix the Error ;**

- If the file is in another folder, give the full/relative path in the script :

```
# Step 5: Map flip-flops to the standard cell library
dfflibmap -liberty osu018_stdcells.lib

# Step 6: Map logic to the standard cell library
abc -liberty osu018_stdcells.lib
```

- Or move the `.lib` into your working directory.
- Re-Run the Script

```
=== counter_4bit_up ===
Number of wires:          24
Number of wire bits:      61
Number of public wires:   3
Number of public wire bits: 6
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:         14
DFFSR                     4
INVX1                     5
NAND2X1                   1
NAND3X1                   1
XNOR2X1                   2
XOR2X1                    1

Removed 0 unused cells and 14 unused wires.

10. Executing Verilog backend.
Dumping module `counter_4bit_up'.

11. Generating Graphviz representation of design.
Writing dot description to '/home/coe-12/.yosys_show.dot'.
Dumping module counter_4bit_up to page 1.
Exec: { test -f '/home/coe-12/.yosys_show.dot.pid' && fuser -s '/home/coe-12/.yosys_show.dot.pid'; } || ( echo $$ >&3; exec xdot '/home/coe-12/.yosys_show.dot'; )
.yosys_show.dot.pid' &

End of script. Logfile hash: 4a499d40ac
CPU: user 0.05s system 0.00s, MEM: 18.38 MB total, 12.45 MB resident
Yosys 0.9 (git sha1 1979e0b)
Time spent: 29% 1x techmap (0 sec), 11% 4x opt_clean (0 sec), ...
```

- Use Command `cat synth.log // for opening the log`



coe-12@ip-10-100-19-29:~/Desktop/Lab/Lab/Understanding_log_file\$ cat synth.log

```
-----
yosys -- Yosys Open SYnthesis Suite

Copyright (C) 2012 - 2019 Clifford Wolf <clifford@clifford.at>

Permission to use, copy, modify, and/or distribute this software for any
purpose with or without fee is hereby granted, provided that the above
copyright notice and this permission notice appear in all copies.

THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES
WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF
MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR
ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN
ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF
OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.
-----

Yosys 0.9 (git sha1 1979e0b)

-- Executing script file `script.js' --

1. Executing Verilog-2005 frontend: design.v
Parsing Verilog input from `design.v' to AST representation.
Generating RTLIL representation for module `counter_4bit_up'.
Successfully finished Verilog frontend.

2. Executing HIERARCHY pass (managing design hierarchy).

2.1. Analyzing design hierarchy..
Top module: counter_4bit_up
```

- Yosys executes the commands inside script.js (reading RTL, synthesis passes, mapping).
- All messages printed by Yosys (info, warnings, errors) are saved into synth.log using the tee command.

Step 5: View Output File (Netlist)

```
gvim design_synth.v
```

OBSERVATION:

- When the Verilog RTL contains syntax errors, Yosys immediately halts the synthesis process and prints descriptive error messages in the log.
- Common issues include missing semicolons, brackets, or mis-declared signals.
- Fix The Library Path

- After fixing the Rtl,Library Path, the synthesis log shows successful execution of passes like proc, opt, fsm, memory, and abc.
- A clean gate-level netlist is only generated after all errors are resolved.
- The log acts as both an **error detector** and a **step-by-step guide** to what the tool is doing internally.

OUTPUT:

- **Yosys log file (synth.log)** containing error messages for the faulty RTL, library path and detailed synthesis passes after correction.
- **Gate-level netlist (design_synth.v)** generated after successful synthesis.

CONCLUSION:

This experiment highlights the importance of reading and interpreting Yosys logs for debugging synthesis issues. By intentionally creating and correcting errors in RTL code, and fixing library path students gain practical experience in identifying syntax issues and understanding how they affect the synthesis process. Mastering log analysis ensures smoother design flows and prepares students for handling complex projects where efficient debugging is critical.